AI Foundations: VS Code + R

Nicholas Warren

Lead, Artificial Intelligence & Machine Learning program, DISSC

nick.warren@yale.edu

Maurice Dalton

Lead, Data Engineering and Solution Team, DISSC

Prerequisites

Complete the setup instructions before starting.

Goals

- Learn how Claude Code can support your research workflow in R
- Practice using AI to explain, iterate on, and extend a real statistical analysis
- Understand the end-to-end setup so you can use these tools independently after the workshop

Workshop outline

Phase	Description	Duration
Overview	Data security and tool orientation	~10 min
1. Concept & idea generation	Explore the research question with Claude Code	~10 min
2. Script creation	Import data and build analysis code	~20 min
3. Analysis & visualization	Iterate on plots, scales, and models	~30 min
4. Debugging	Diagnose and fix broken code	~20 min

How it works

In this workshop you use **VS Code** as your editor with the **R** and **Quarto** extensions, and **Claude Code** as your AI assistant. Claude Code is a command-line tool made by Anthropic that brings Claude directly into your terminal.

Your prompts and code are sent to **Claude Opus 4.6** through **AWS Bedrock**, Amazon's managed AI service, running on Yale's AWS account.

VS Code for RStudio users

If you're coming from RStudio, here's where to find the features you know:

RStudio	VS Code equivalent
Console	**R Terminal** (bottom panel)
Source editor	**Editor** (center). Run lines with `Cmd + Enter` / `Ctrl + Enter`
Environment pane	**R: Workspace** in the sidebar (click the R icon)
Plots pane	Renders inline in the Quarto notebook
Help pane	Hover over a function, or `?function_name` in the R Terminal
Viewer / data	`View(dataframe)` in the R Terminal

What is Claude Code?

Claude Code is Anthropic's official command-line tool for Claude. It runs in your terminal and can:

- **Read and understand your project files** without copying and pasting
- **Edit files directly** when you ask it to make changes
- **Run commands** like R scripts, git operations, and more
- **Answer questions** about your code, data, and analysis

Data security

- **Your data is not used for training.** AWS Bedrock does not use your prompts or outputs to train any models.
- **Your data is not shared.** Model providers (including Anthropic) do not have access to your inputs, outputs, or logs.
- **Enterprise compliance.** AWS Bedrock is HIPAA eligible, SOC and ISO certified, and GDPR compliant.

In short: you get the benefits of a frontier LLM without research data leaving Yale's AWS environment or being used to improve future models.

Open the workshop notebook

- In VS Code, open `files/code/example.qmd` from the file explorer
- This is a **Quarto notebook** that mixes markdown text with R code chunks
- Click the **Run Cell** button above any code chunk to run it
- Output (plots, tables, text) appears inline below each chunk

Start Claude Code

Open a terminal in VS Code (`Ctrl + `) and run:

```
claude
```

This opens an interactive session. Useful commands:

Command	Action	Command	Action
`/help`	Show commands	`/model`	Switch models
`/clear`	Clear history	`Ctrl + C`	Cancel response
`/exit`	Exit Claude Code	`Ctrl + D`	Exit Claude Code

Two terminals, two tools

VS Code's bottom panel can have multiple terminals open at once. In this workshop you'll use two:

- **R Terminal**: runs R code. Created automatically when you open a `.qmd` or `.R` file. Use it for `install.packages()`, `library()`, and running code chunks.
- **Terminal**: a regular shell (zsh/bash/cmd). Open one with `Ctrl + ` ` . This is where you run `claude` to start Claude Code.

You can switch between them using the dropdown in the terminal panel header.

Tips for working with Claude Code

- **Be specific.** "Add clustered standard errors at the state level" works better than "improve the model."
- **Point to files.** Claude Code can read your project, but referencing a file by name (e.g., "in example.qmd") focuses its attention.
- **Iterate.** If the first result isn't right, follow up: "That's close, but use `sandwich` instead of `estimatr`."
- **Ask it to explain.** "Why did you use that approach?" is always a good follow-up.

Review before you accept

Claude Code will ask for permission before editing files or running commands. Before approving:

- **Read the diff.** Claude Code shows you exactly what it plans to change. Make sure you understand the edit before accepting.
- **Run the code yourself.** After Claude edits a chunk, run it to confirm it produces the output you expect.
- **Verify statistical claims.** AI can make reasoning errors. Always check coefficients, p-values, and interpretations against your own understanding.

> You can type `!` to allow all actions for the session, but we recommend reviewing individually while you're learning.

Phase 1: Concept & Idea Generation

Use Claude Code to explore the research question **before writing any code**.

Try these prompts:

> What is Difference-in-Differences and when is it appropriate to use?

> I have organ donation rate data for 27 states across 6 quarters. California introduced an "active choice" policy in Q3 2011. How would I set up a DiD analysis?

Phase 2: Script Creation

Ask Claude Code to write the import code. It can edit the notebook directly.

> Read in data/organ_donations.csv and prepare it for a DiD analysis. Write the code in the Phase 2 section of files/code/example.qmd.

Run the chunks Claude creates to verify they work.

Phase 2: About the data

The dataset (`data/organ_donations.csv`) has 162 rows: 27 states across 6 quarters (Q4 2010 through Q1 2012). California's "active choice" organ donation policy took effect in Q3 2011.

Data source: Kessler & Roth (2014), "Don't Take 'No' For An Answer." Dataset via the `causaldata` R package, featured in *The Effect* by Nick Huntington-Klein.

Phase 3: Analysis & Visualization

The notebook has pre-written code for a DiD plot and regression model. Run these chunks, then use Claude Code to iterate:

> Change the y-axis to show values from 0% to 100%

> Switch the plot to a black-and-white theme suitable for print

> Add robust standard errors clustered at the state level

> Create a summary statistics table broken down by treatment group and period

Phase 3: More ideas

> Export the coefficient table as a CSV and the plot as a high-resolution PNG to an output/ folder

> Add a placebo test using a fake treatment date one quarter earlier

> Explain the coefficients from the DiD regression in plain language

See `files/prompts/examples.md` for more prompts.

Phase 4: Debugging

The bottom of the notebook has four code chunks with intentional bugs. Ask Claude Code to find and fix them:

> Look at the Phase 4 section of files/code/example.qmd and fix the bugs

Then try a follow-up:

> For each bug you fixed, explain what went wrong and how a beginner could spot the same mistake

Troubleshooting

- **Claude Code not responding:** Restart your terminal and verify credentials (Step 6 in setup). Run `echo $CLAUDE_CODE_USE_BEDROCK` to confirm it's `1`.
- **R packages not found:** Run `install.packages("tidyverse")` in the R Terminal.
- **No code completion:** Make sure `languageserver` is installed (Step 13 in setup). Restart VS Code.
- **Script errors:** Check the working directory with `getwd()` in the R Terminal.

Next steps

- **Get long-term access.** Your workshop API key will expire. Contact dissc@yale.edu for ongoing access.
- **Try your own data.** Bring a dataset and a research question. Claude Code can help you write and refine the analysis.
- **Explore Claude Code further.** Try asking it to create new scripts, debug errors, or explain unfamiliar code.
- **Get in touch.** nick.warren@yale.edu or dissc@yale.edu